POLARIS
SCHOOL OF TECHNOLOGY

NORMALIZATION AND REGULARIZATION IN NEURAL NETWORKS

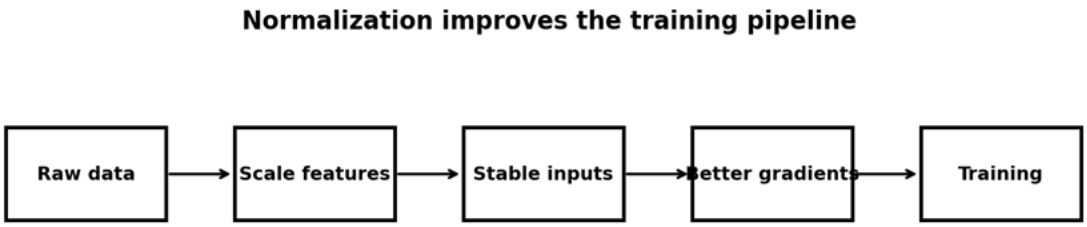
1. Introduction

Normalization and regularization address different problems in neural-network training. Normalization controls the scale or distribution of data and activations, making optimization more stable and efficient. Regularization controls model complexity and reduces overfitting so that the learned model generalizes better to unseen data.

Concept	Main purpose	Examples
Normalization	Improve numerical scale and optimization	Min-max scaling, standardization, BatchNorm, LayerNorm
Regularization	Reduce overfitting / improve generalization	L1, L2, dropout, early stopping, augmentation, weight decay

2. Why Normalization Is Needed

If input features have very different numerical scales, gradient-based optimization can become inefficient. A feature measured between 0 and 1 and another measured in millions can produce very different gradient scales. Scaling makes the optimization problem easier to navigate.



2.1 Standardization

$$z = \frac{x - \mu}{\sigma}$$

Z-score standardization: subtract the training mean and divide by the training standard deviation.

- Centers the feature around zero.
- Produces a standard deviation of approximately one on the training set.
- Often a strong default for continuous numerical features.
- Fit μ and σ using training data only.

2.2 Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Min-max scaling to approximately [0,1].

Min-max scaling is useful when a bounded range is desirable, but it can be sensitive to outliers because extreme values determine the range.

2.3 Image Normalization

$$x' = \frac{x}{255}$$

Common rescaling of 8-bit image pixels from 0–255 to approximately 0–1.

Consistent scaling helps the optimizer operate on manageable numerical values.

2.4 Avoiding Data Leakage

$$x'_{\text{val}} = \frac{x_{\text{val}} - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Validation/test data should use statistics learned from the training set.

Never calculate normalization statistics from the test set and then report test performance as if the test set were completely unseen.

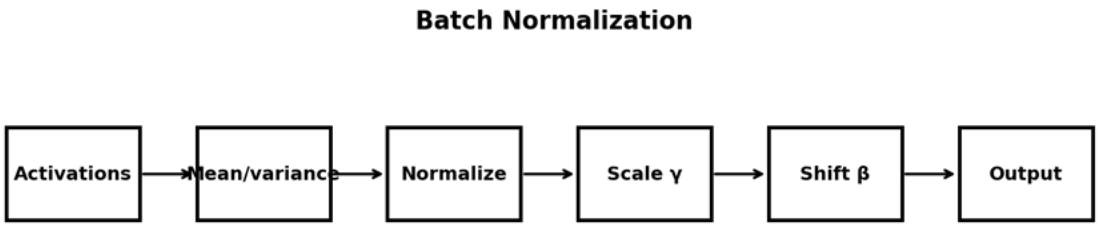
3. Normalization Layers

Normalization can also occur inside a neural network. These layers normalize intermediate activations rather than only preprocessing the original features.

Method	Main idea	Common use
Batch Normalization	Uses mini-batch statistics for each feature/channel	CNNs and many feed-forward networks
Layer Normalization	Normalizes features within each individual sample	Transformers and sequence models
Instance Normalization	Normalizes spatial features per sample/channel	Style transfer and image generation
Group Normalization	Normalizes groups of channels	CNNs with small batch sizes

4. Batch Normalization

Batch Normalization normalizes activations using statistics calculated from a mini-batch and then applies learnable scale and shift parameters.



$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

Mini-batch mean.

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_B)^2$$

Mini-batch variance.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

Normalized activation.

$$y_i = \gamma \hat{x}_i + \beta$$

Learnable scale γ and shift β.

- During training, batch statistics are used.
- Moving statistics are maintained for inference in typical BatchNorm implementations.
- ε prevents division by a value that is zero or extremely small.
- γ and β preserve the layer's ability to learn useful scales and offsets.

5. Layer Normalization

Layer Normalization calculates statistics within each individual sample. It does not depend on the batch in the same way BatchNorm does, making it particularly useful for transformers and sequence models.

$$\mu = \frac{1}{H} \sum_{j=1}^{H} x_j$$

Mean across H features of one sample.

$$\sigma^2 = \frac{1}{H} \sum_{j=1}^{H} (x_j - \mu)^2$$

Variance across those features.

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

Normalized feature.

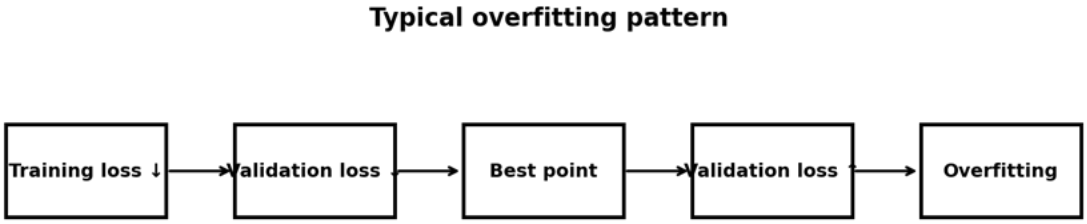
$$y_j = \gamma_j \hat{x}_j + \beta_j$$

Learnable scale and shift.

BatchNorm	LayerNorm
Relies on batch statistics during training	Works independently for each sample
Can be affected by very small batches	Naturally handles small/variable batch sizes
Common in CNN workflows	Common in transformers

6. What Is Regularization?

Regularization is a set of methods that discourages a neural network from fitting noise or memorizing the training set. The desired result is better performance on unseen data.



Condition	Training	Validation/Test
Underfitting	Poor	Poor
Good generalization	Good	Good
Overfitting	Very good	Worsening / poor

7. L1 Regularization

L1 regularization adds the absolute value of weights to the loss. It tends to encourage sparse parameter values.

$$\mathcal{L} = \mathcal{L}_{data} + \lambda \sum_i |w_i|$$

L1-regularized objective.

$$\frac{\partial}{\partial w}|w| = \text{sign}(w) \quad (w \neq 0)$$

Gradient/subgradient away from zero.

- Main characteristic: encourages sparsity.
- Useful when many weights being near zero is desirable.
- Large λ can remove too much capacity and cause underfitting.

8. L2 Regularization

L2 regularization penalizes squared weights and generally shrinks large weights without forcing as many weights exactly to zero.

$$\mathcal{L} = \mathcal{L}_{data} + \frac{\lambda}{2}\sum_i w_i^2$$

L2-regularized objective.

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}_{data}}{\partial w_i} + \lambda w_i$$

L2 adds λw to the gradient.

- Large weights receive stronger shrinkage.
- Produces smoother weight distributions than L1.
- Widely used as a general-purpose regularization mechanism.

9. L1 vs L2

Feature	L1	L2
Penalty	|w|	$w^2/2$
Main effect	Sparsity	Weight shrinkage
Exactly zero weights	More likely	Less likely
Smooth at zero?	No	Yes
Typical purpose	Sparse/feature selection	General weight control

10. Elastic Net

Elastic Net combines L1 and L2 regularization.

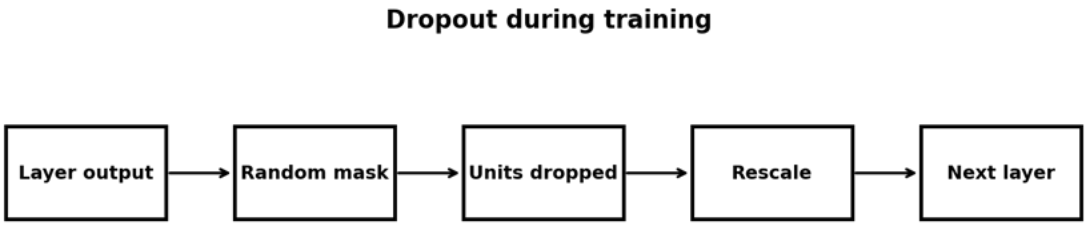
$$\mathcal{L} = \mathcal{L}_{data} + \lambda_1 \sum_i |w_i| + \frac{\lambda_2}{2} \sum_i w_i^2$$

Elastic Net objective.

It provides a compromise between sparsity and smooth weight shrinkage.

11. Dropout

Dropout randomly disables a fraction of activations during training. It prevents the network from becoming excessively dependent on particular neurons and encourages distributed representations.



$$\tilde{x}_i = m_i x_i, \qquad m_i \sim \text{Bernoulli}(1-p)$$

Conceptual dropout mask; p is the drop probability.

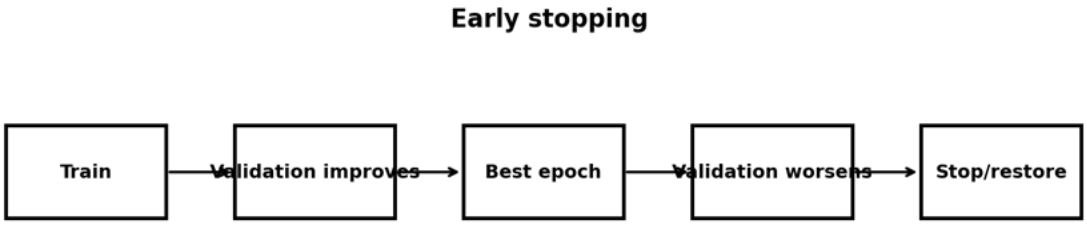
$$\tilde{x}_i = \frac{m_i x_i}{1-p}$$

Inverted dropout scaling during training.

- Dropout is normally disabled during inference.
- Too much dropout can cause underfitting.
- Dropout is stochastic: different training updates can use different masks.

12. Early Stopping

Early stopping terminates training when validation performance stops improving. The model can retain the parameters from the best validation epoch.



- Use a validation set to decide when to stop.
- Patience allows temporary fluctuations before stopping.
- Do not use the test set repeatedly to choose the stopping point.

13. Data Augmentation

Data augmentation creates realistic variations of training examples. In image tasks, appropriate transformations can include small rotations, crops, translations, flips, or brightness changes. The transformation must preserve the target label.

- Increases effective diversity of training data.
- Can reduce overfitting.
- Especially useful when the original dataset is limited.
- Incorrect transformations can create mislabeled examples.

14. Weight Decay

Weight decay directly shrinks parameters during optimization. For simple SGD, its relationship with L2 regularization is especially close.

$$w_{t+1} = w_t - \eta g_t - \eta \lambda w_t$$

SGD update with weight decay.

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta g_t$$

Same update written as direct parameter shrinkage plus gradient step.

With adaptive optimizers such as Adam, decoupled weight decay is preferable when the intention is to apply a direct decay mechanism; this is the motivation behind AdamW.

15. Normalization vs Regularization

Question	Normalization	Regularization
Primary objective	Stable scale / optimization	Better generalization
Typical examples	Standardization, BatchNorm, LayerNorm	L1, L2, dropout, early stopping, augmentation, weight decay
Does it prevent overfitting?	Not necessarily	Yes, by design
Can they be combined?	Yes	Yes

Normalization and regularization are complementary. A model can be normalized and regularized at the same time.

16. How to Detect Overfitting

The most useful signal is often a divergence between training and validation behavior: training loss keeps falling while validation loss stops improving or starts rising.

$$\text{Overfitting signal:} \quad L_{train} \downarrow \quad \text{while} \quad L_{val} \uparrow$$

Typical qualitative overfitting pattern.

- Compare both losses, not only training accuracy.
- Check validation and test performance.
- Try regularization only after establishing a clear baseline.

17. Practical Selection Guide

Problem	Possible starting point	Reason
Continuous features have very different scales	Standardization	Creates comparable feature scales
Image pixels are 0–255	Rescale to 0–1 or appropriate model preprocessing	Keeps input values numerically manageable
CNN with ordinary batch sizes	BatchNorm	Effective activation normalization
Transformer / sequence model	LayerNorm	Independent of batch statistics
Overfitting in dense network	L2, dropout, early stopping	Controls model complexity
Need sparse weights	L1	Encourages many small/zero weights